Chapitre 8 :
C

Table des matières

N.1 Introduction : *Particularités de C* 2

I Rappels de compilation 3

II Les bases du standard C 4

II.1 Composants syntaxiques de bases 4
II.2 Types et constantes 5
II.3 Opérateurs usuels 5
II.4 Instructions usuelles 8

III Pointeurs 10

III.1 Organisation de la mémoire 10
III.2 Adresses 10
III.3 Allocation dynamique 12

IV Les « tableaux » 13

IV.1 Tableaux statiques unidimensionnels 13
IV.2 « Tableaux » dynamiques 14
IV.3 Tableaux multidimensionnels 15
IV.4 Chaînes de caractères 15
IV.5 Structures 16

N.1 Introduction : *Particularités de C*

Le typage en C

C dispose d'un typage **fort**, **dynamique**

- Un typage **fort** signifie que que le type est détecté à la compilation.
- Un typage **synamique** signifie qu'une variable peut changer de type implicitement si besoin

Paradigme de programmation

C est un langage **impératif**

Informations diverses

C est un langage **compilé**

La gestion de la mémoire est laissée en partie à la charge du développeur

C est un langage **strict**, les arguments d'une fonctions sont alors évalués avant son appel.
(Ce n'est pas le cas par exemple du Haskell qui va évaluer les arguments uniquement lorsqu'ils sont nécessaires).

C est un langage **impur**, il est alors possible de modifier la valeur d'une « variable » (toujours en opposition au Haskell)

I Rappels de compilation

Définition

La **compilation** est le processus permettant de créer un **exécutable** à partir d'un **code source**. Il comporte généralement 4 phases

1. Le traitement par le préprocesseur : manipulations textuelles (copie des fichiers `#include` dans le code source, remplacement des directives `#define`, etc…)
2. La compilation : traduction du code source en assembleur
3. L'assemblage : traduction de l'assembleur en binaire, produit un fichier `.o`
4. L'édition de liens : lie plusieurs fichiers objet pour faire un seul exécutable

Syntaxe de compilation

```
gcc <options de compilation> <fichier>.c <liaisons de bibilothèque>
```

On utilise par exemple comme options :

- `-g` : donne des informations de debug
- `-Wall` et `-Wextra` : explicitent les avertissements
- `-fsanitize=address` : explicite les problèmes d'utilisation de mémoire à l'exécution
- `-o <nom>` : renommage du ficher créé par gcc
- `-pthread` : permet d'utiliser la bibilothèque `<pthread.h>` (multithreading)
- `-c` : interrompt la compilation juste après l'assemblage et produit les fichiers objet

Exemple de lisaisons de bibliothèque :

- `-lm` : permet de faire des maths

Par exemple on utilisera une commande proche de
`gcc -g -Wall -Wextra - fsanitize=address -o test test.c -lm`

Un exemple d'utilisation de `-c` est de ne pas recompiler l'ensemble des dépendances et le fichiers final si une seule d'entre elle est modifiée.

II Les bases du standard C

II.1 Composants syntaxiques de bases

Il y a 6 types d'unités lexicales en c

Les identificateurs

Un **identificateur** donne un nom aux objets manipulés par le programme.

Un identificateur est une chaîne finie de caractères `{a, ... , z, A, ... , Z, 0, ... , 9, _}` qui ne commence par par un chiffre

Les mots clés du langage

`auto, register, static, extern, typedef` ← explicitent comment sont stockés les informations dans la mémoire
`char, double, float, int, enum, long, short, signed, unsigned, union, void, struct` ← définissent des types
`const, volatile` ← qualificateurs de type
`break, case, continue, default, do, else, for, goto, if, switch, while, return, sizeof` ← mots clés de « contrôle »

Ces mots clés ne peuvent pas être utilisés comme identificateurs

Les constantes

Par exemple 1, 17.3, 'a'

Les chaînes constantes

Par exemple "arg"

Les opérateurs

Par exemple `->, =, ++, &&, ...`

La ponctuation

`{, }, ,, (, )`

II.2 Types et constantes

1. Pour les entiers, il y a le type de base `int` ainsi que de nombreux types dérivés, par exemple `unsigned int`, `long int`, `short int`, ... La différence entre ces types est le nombre de bits utilisés pour représenter l'entier. **Il est dépendant, au sein d'un même type, de la machine et peut-être du compilateur**

   Pour résoudre ce problème, la bibliothèque `<stdint.h>` fournit des types entiers avec garanties : `int32_t`, `int_64_t` pour les entiers signés, `uint32_t`, `uint64_t` pour les non signés

2. Pour les flottants, il existe `float`, `double`, `long double`

3. Les caractères, ils sont entre `'` Les caractères sont des entiers mais on fait comme si tkt

4. Les chaîne de caractères constantes, entre `"`

II.3 Opérateurs usuels

L'affectation :

`<lvalue> = <expr>;`
Évalue `<expr>` et la stocke dans `<lvalue>`

Remarque

L'affectation peut faire des conversions de type implicites de sorte que `<expr>` ait le bon type vis à vis de `<lvalue>`. Par exemple,

```
int i = 0;
int j = 2;
float x = 2.5;
i = j+x; // Ici, on obtiendra 4 au lieu de 4.5
```

Les opérations arithmétiques

Pour +, -, *, tout va bien
Attention, / opère la division entière si les arguments sont entiers, et la division flottantes si les **arguments** sont floattants (au moins un argument)

Par exemple,

```
float x = 3/2; // Contient 1
float x = 3/2.; // Contient 1.5
```

Pour le %, dans la norme, on a toujours `b*(a/b) + a % b = a`

> ⚠ **Avertissement**
>
> Il n'y a pas d'exponentiation nativement, il faut charger la bibilothèque math et utiliser la fonction pow (include math.h et rajouter -lm à la fin)

Les opérateurs relationnels

`<exp1> op <exp2>` avec pour opérateurs, {<, >, ⩽, ⩾, ≠, =}

Les opérateurs logiques

! donne la négation
&& et || donnent le et et le ou et sont paresseux

Les opérateurs de conversion de type explicite

`(<type>)<objet>`

Dans le cas particulier du `(type)malloc( ... );`, il est facultatif en C mais recommandé par le programme (il est d'ailleurs obligatoire en C++)

Cas particulier : le cas des caractères
Si `c` est un `char`, alors `(int)c` permet, d'après le programme « d'accéder au code ASCII de c »

Point Culture

Les opérateurs bit à bit :

`|` : ou, `&` : et, `~` : no, `<<` : décalage à gauche, `>>` : décalage à droite , `^` : xor

Les opérateurs d'affectation composée

`<e1> op= <e2>` : pareil que `<e1> = <e1> op <e2>`
On a `+=`, `-=`, `*=`, `/=`, `%=`

Opérateurs d'incrémentation/décrémentation

++ et --

L'opréteur d'adressage

cf. Partie sur la mémoire

II.4 Instructions usuelles

Déclaration

`<type> <nom>;` pour une déclaration
`<type> <nom> = <valeur>;` pour une déclaration et une affectation

Remarque

On peut déclarer simultanément plusieurs objets, via par exemple

```
int i, j;
int i = 0, j;
```

Avertissement

Par défaut, toutes les variables sont modifiables. Pour empêcher ce comportement, on utilise le mot-clé `const`. `const` s'applique à ce qui est à sa gauche, sauf s'il n'y a rien à sa gauche auquel cas il s'applique à sa droite.

Par exemple

```
const int n = 3;
const int* p = &n; // p est un pointeur (pas constant) vers un
entier constant
*p = 42; // INTERDIT
p = &m; // Est parfaitement autorisé, que m soit constant ou non, et
dans tous les cas il n'est pas possible de modifier m en utilisant
le pointeur

int n = 3;
int* const p = &n; // p est un pointeur constant sur un entier (pas
constant)
*p = 42 // Fonctionne
p = &m; // INTERDIT

const int* const p; // p est un pointeur constant sur un entier
constant
```

Les assertions

```
assert (<condition>);
```

Les instructions composées

```
{<déclarations locales>; <instr1>; <instr2>; ... }
```

Branchements conditionnels

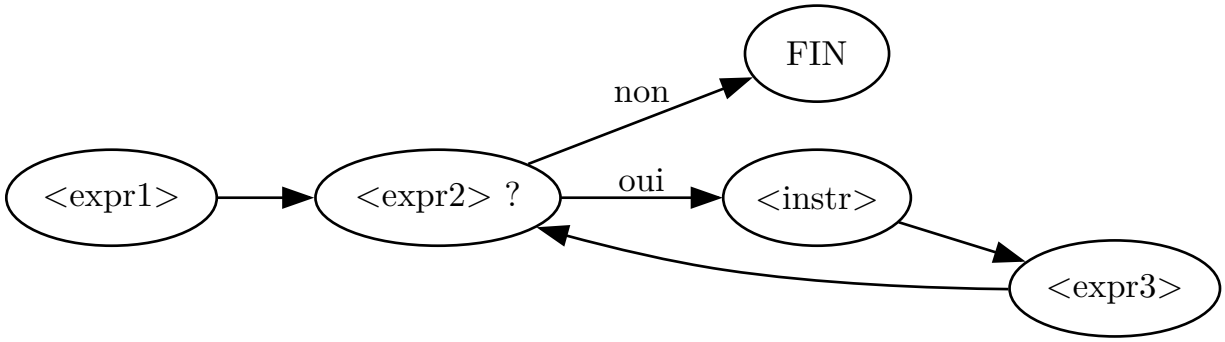
```
if (<expr1>) <instr1> else <instr2>
```

Par exemple, `if (random() % 2){ ... }` est licite, celà correspond alors à `if (random() % 2 ≠ 0){ ... }`
Par exemple, `if (ptr) { ... }` est l'équivalent de `if (ptr ≠ NULL) { ... }`

En cas de non parenthésage, le else se raccroche toujours au if le plus proche

Boucles for et while

```
for (<expr1> ; <expr2> ; <expr3>) <inst>
```



`while (<expr>) <instr>`
Tant que `<expr>`, on fait l'instruction

> 🔥 ***Remarque***
>
> `for (e1; e2; e3) {i;}` est équivalent à
>
> ```
> {e1;
> while (e2){
>  i;
>  e3;
> }}
> ```

Branchements non conditionnels

`break` interrompt une boucle et la fait passer à l'instruction qui suit immédiatement la boucle.
`continue` fait passer à l'itération suivante

III Pointeurs

III.1 Organisation de la mémoire

La mémoire **virtuelle** d'un processsus est divisée en :

- Une zone pour les valeurs statiques initialisées
  Par exemple, `int x = 3;` en variable globale
- Une zone pour les valeurs statiques non initialisée
  Par exemple, `int x;` en variable globale
- La pile qui contient des blocs d'activation stockant les appels de fonction en cours et leurs variables locales
- Le tas qui sert à allouer des objets dynamiquement

🔥 ***Remarque***

On parle de mémoire virtuelle par opposition à la mémoire physique, comprenant par exemple les registres, les caches, la RAM, la mémoire de masse, etc…

La machine décide dynamiquement où elle stocke la mémoire virtuelle

III.2 Adresses

ℹ **Définition**

Un **pointeur** est un objet dont la valeur est une adresse d'un autre objet

La **valeur de p** est l'adresse qui est sa valeur / le **contenu de p** est la valeur de l'objet pointé par p

En C, on peut explicitement créer des objets qui sont des pointeurs (contrairement au OCaml)

Syntaxe des pointeurs

Si `<t>` est un type, `<t>*` est le type des pointeurs sur les objets de type `<t>`

Par exemple, `int* p, int *p et int * p` permettent des déclarer des pointeurs sur un entier.

Avertissement

`int* a, b;` déclare `a` comme un pointeur sur un entier et `b` comme un entier.
Pour déclarer deux pointeurs il faut utiliser `int *a, *b;` ou simplement

```
int* a;
int* b;
```

Pour affecter une valeur à un pointeur on peut:

1. Utiliser la constante littérale `NULL`, par exemple
   ```
   int* p  = NULL
   ```
2. Récupérer l'adresse d'un objet déjà déclaré
   Par exemple, si `a` est un objet, `&a` est l'adresse de `a`, on peut donc faire :
   ```
   int a;
   int* p = &a;
   ```

Pour procéder au déréférencement d'un pointeur, on utilise `*`.
Pour accéder au contenu du pointeur `p`, on utilise donc `*p`
`*p` est une `lvalue`, `*p = 3;` est donc correct.

Avertissement

Déréférencer `NULL` est une undefined behavior

3. On demande au système une adresse avec `malloc`

Remarque

En C, tous les arguments sont passés par valeur, c'est à dire que la fonction agit sur des copies locales des valeurs en argument. Mais ces valeurs peuvent être des adresses et dans ce cas, abusivement, on dit que les arguments sont passés par valeurs.

III.3 Allocation dynamique

Syntaxe

Après avoir `#include <stdlib.h>`

`void* malloc(size_t size)` où `void*` est un type pointeur générique.

L'entrée de `malloc` est la taille à allouer en octets, on obtient alors en sortie l'adresse du début du bloc alloué.

Remarque

`malloc` peut échouer, auquel cas il renvoie le pointeur `NULL`

Pour obtenir la taille d'un objet et allouer un bloc de la bonne taille, on utilise l'opérateur `sizeof` qui prend en argument un type et renvoie sa taille en octets. Par exemple, `sizeof(char) = 1`, mais `sizeof(int)` est implémentation et machine dépendant.

Pour libérer la mémoire allouée par `malloc`, on utilise `void free(void* ptr)`

Avertissement

1. On ne peut `free` que des objets créés par `malloc`

```
int a = 3;
int* p = &a;
free(p); // INTERDIT
```

2. On ne peut évidemment plus accéder au contenu d'un pointeur libéré
3. On ne peut `free` qu'une fois

Remarque

`free(NULL);` est parfaitement valide

IV Les « tableaux »

IV.1 Tableaux statiques unidimensionnels

Syntaxe

`<type des éléments du tableau> <nom>[<taille>];`
La taille doit être conne avant la compilation.

Par exemple, `int tab[5];` est correct mais `int n = 5; int tab[n];` ne l'est pas
De plus,

```
#define n 5
int tab[n];
```

est possible.

Aussi, `tab[i]` est une `lvalue`

Accéder à la i-ième case de `tab` se fait avec `tab[i]` (numérotation à partir de 0)
Si on accède à une case en dehors du tableau le comportement est indéfini (UB)

Attention, les tableaux ne stockent pas leur taille, on a ainsi plusieurs possibilités :
- à chaque fois qu'on donne un tableau en entrée d'une fonction on donne aussi sa taille
- faire une `struct` stockant la taille
- mettre une sentinelle à la fin du tableau

Intialisateurs

`int tab[5] = {1, 2, 3, 4, 5};`
Quand on utilise un initalisateur, il n'est pas nécessaire de mettre la taille :
`int tab[] = {1, 2, 3, 4, 5};`
On peut aussi faire `int tab[10] = {1, 2, 3}` qui va initialiser les premières cases avec les valeurs données et intialiser les autres avec 0. On peut ainsi utiliser `int tab[100] = {0}` pour se passer d'une boucle `for` d'initialisation.

Avertissement

Un initialisateur n'est pas un tableau, il faut y penser comme un alias pour une boucle `for` qui remplie le tableau. Il n'est d'ailleurs utilisable qu'à la déclaration.

IV.2 « Tableaux » dynamiques

Avertissement

Le terme de tableau dynamique peut désigner aussi bien un tableau alloué sur le tas comme à une structure de donnée de tableau redimensionnable

Syntaxe

```
<type>* <nom> = malloc(sizeof(<type>)*<taille>);
```

Cela ne crée pas vraiment un tableau mais un pointeur vers une zone mémoire qui **se comporte comme** un tableau

Pour la différence, cf. code Nextcloud

La confusion entre « vrai tableau » et « pointeur qui se comporte comme un tableau » vient du fait qu'ils se manipulent pareil.

Par exemple, si on a un fonction qui est censé prendre en argument des tableaux statiques, on peut parfaitement lui donner un tableau dynamique en arguments. De même dans l'autre sens. Ainsi, voici un code parfaitement correct :

```
int elem_stat(int t[]){
  return t[0];
}

int elem_dyn(int* t){
  return t[0];
}

int* t = malloc(sizeof(int)*2);
t[0] = 1; t[1] = 36;
elem_stat(t);
free(t);

int t[3] = {1, 42, 8};
elem_dyn(t);
```

IV.3 Tableaux multidimensionnels

Syntaxe des matrices

Pour manipuler une matrice à `n` lignes et `m` colonnes, on a plusieurs options :

1. Allocation dynamique d'un tableau de tableaux

```
int** mat = malloc(sizeof(int*)*n);
for (int i = 0; i < n; i++){
  mat[i] = malloc(sizeof(int)*n);
}
```

2. Allocation dynamique en linéarisant

```
int* mat = malloc(sizeof(int)*n*m);
mat[i*m + j]; // Accède à la case [i][j]
```

3. De manière statique

```
int mat[10][4];
```

   Attention, toutes les dimensions doivent être spécifiées, sauf pour la première (y compris dans les arguments des fonctions) : `int mat[][4]`
   On peut faire : `int mat[][2] = {{1,2},{3,4},{5,6}};`

IV.4 Chaînes de caractères

Idée

Une chaine de caractère se comporte comme un tableau de caractères se finissant par `'\0'`

Déclaration des chaînes de caractère

1. En statique : `char s[0] = {'t', 'o', 't', 'o', '\0'}`
2. En statique avec initialisation avec une chaîne : `char s[0] = "une chaine";`
3. En dynamique : `char* s = malloc(sieof(char)*taille)` avec une taille suffisante pour mettre le `'\0'` On peut alors la remplir avec `strcpy(s, "chaine")`

Remarque

`char* s = "toto";` ne provoque pas d'erreur mais dans ce cas `s` est non modifiable. En effet, écrire `s[0] = 'a';`, cela revient exactement à écrire : `"toto"[0] = 'a';`

IV.5 Structures

Un peu de syntaxe

```
struct <nom> {
  <type1> <nom1>;
  ...
  <typek> <nomk>;
};
```

Par exemple :

```
struct frac {
  float num;
  float denom;
}

// Dans le main :
struct frac demi = {.num = 1., .denom = 2}
```

On peut renommer le type avec :

```
typedef  <ancien nom> <nouveau nom>;
typedef struct frac frac; //par exemple
```

On peut également directement déclarer et renommer en même temps

```
typedef struct frac { ... } frac;
```

On peut également accéder aux champs des structures déclarées oO

```
<nom>.<nom du champ>;
<nom>.<nom du champ> =  ... ;
```

On peut initialiser en remplaçant à la main ou avec un initialisateur :

```
<type> <nom> = {.<champ1> = <valeur1>,  ... , .<champk> = <valeurk>};
```

Cependant, la métiode de l'initialisateur n'est plus utilisable si on manipule un pointeur sur la structure plutôt que la structure.

Quand on manipule une structure par le biais d'un pointeur et qu'on veut accéder à un champ, on peut utiliser

```
(*<objet>).<champ>;
<objet>→<champ>;
```